

Full-Stack Web Development — Day 10 Study Material

MODULE	SUBJECT	AUTHOR
State Management & Data Architecture	TanStack React Query & Zustand	Penumuru Madhu Sudhan Reddy

Executive Summary: Comprehensive architectural guide to separating server state from client UI state. Explores TanStack React Query for background caching, request deduplication, and optimistic mutations with rollback, alongside Zustand for client-only UI state and token persistence.

1. Server State vs. Client State Architecture

Dimension	Server State (TanStack React Query)	Client UI State (Zustand)
Location	Originates on backend DB; cached locally	Local browser runtime memory only
Nature	Asynchronous, shared, potentially stale	Synchronous, session-scoped
Examples	Project lists, site telemetry, tower specs	Active modal, filter selection, sidebar state
Key Capabilities	Automatic deduplication, stale-while-revalidate	Micro-bundle (1.2KB), no provider wrapping

2. TanStack React Query (useQuery & useMutation)

Declarative data fetching replaces imperative useEffect lifecycle hooks:

```
// Declarative query hook
export function useProjects() {
  return useQuery({
    queryKey: ['projects'],
    queryFn: fetchProjects,
  });
}
```

Benefits in NirmanIQ Operations:

- Zero redundant fetches when multiple dashboard widgets request project metrics.
- Background synchronization keeps site telemetry up-to-date automatically.

3. Optimistic Updates & Cache Rollback (Exercise 2)

Provides instantaneous user feedback by updating the cache ahead of server response:

```
export function useCreateProject() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: createProjectApi,
    onMutate: async (newProject) => {
      await queryClient.cancelQueries({ queryKey: ['projects'] });
      const previous = queryClient.getQueryData(['projects']);
      queryClient.setQueryData(['projects'], [newProject, ...previous]);
      return { previous }; // Snapshot for rollback
    },
    onError: (err, newProject, context) => {
      queryClient.setQueryData(['projects'], context.previous);
    },
    onSettled: () => {
      queryClient.invalidateQueries({ queryKey: ['projects'] });
    }
  });
}
```

4. Zustand Client State Management (Exercise 3)

Direct state slices with minimal boilerplate and built-in persistence:

```
export const useUIStore = create((set) => ({
  viewMode: 'grid',
  statusFilter: 'all',
  setViewMode: (mode) => set({ viewMode: mode }),
  setStatusFilter: (filter) => set({ statusFilter: filter }),
}));
```